VYOMANETRA

AI-Based Drone Detection System Using YOLOv8

Project Report

Submitted by:
Vangapalli Kaveri

Technologies:
Python | YOLOv8 | PyTorch | OpenCV | GitHub

Year: 2026

CERTIFICATE

This is to certify that the project entitled

"Vyomanetra – AI-Based Drone Detection System Using YOLOv8"

has been successfully completed by

Vangapalli Kaveri

as part of self-directed learning and practical implementation
of Artificial Intelligence and Computer Vision technologies.

_____

Vangapalli Kaveri
Author

DECLARATION

I hereby declare that the work presented in this project,
"Vyomanetra – AI-Based Drone Detection System Using YOLOv8"
is my original work and has been carried out using publicly
available tools, datasets, and learning resources.

_____

Vangapalli Kaveri
Author

TABLE OF CONTENTS

1. Executive Summary

Vyomanetra is an Artificial Intelligence-based Drone Detection System developed using the YOLOv8 object detection framework and Python. The system is engineered to detect unmanned aerial vehicles (UAVs) in static images, pre-recorded video streams, and real-time webcam feeds, employing deep learning and computer vision techniques to achieve high detection accuracy and operational efficiency.

The primary objective of this project is to demonstrate the practical application of modern object detection algorithms within surveillance and monitoring scenarios. By employing a custom-labelled drone dataset and GPU-accelerated model training on an NVIDIA RTX 3050 graphics processor, the system achieves reliable and scalable drone detection capabilities. The project establishes a strong technical foundation for further research and industrial deployment in the domains of security, defence, and critical infrastructure protection.

2. Introduction

The rapid proliferation of unmanned aerial vehicles (UAVs), commonly referred to as drones, has fundamentally transformed numerous industries, including logistics, precision agriculture, aerial photography, environmental monitoring, and defence. The global UAV market continues to expand at an accelerated pace, with millions of commercial and consumer drones now operating worldwide.

Whilst drones offer considerable operational advantages, their unrestricted use within sensitive or restricted airspace presents significant security and privacy challenges for government agencies, military establishments, airports, and critical infrastructure operators. Unauthorised drone incursions have been recorded at airports, correctional facilities, and government installations, highlighting the urgent requirement for automated detection solutions.

Traditional monitoring systems rely predominantly upon human observation, rendering them susceptible to operator fatigue, delayed response times, and coverage limitations. Artificial Intelligence, in particular deep learning-based object detection, provides a scalable and reliable solution by automating the identification and localisation of drones within visual data streams.

This report presents Vyomanetra, an intelligent drone detection system that leverages the capabilities of the YOLOv8 architecture to automatically detect drones across diverse visual inputs.

3. Problem Statement

The increasing accessibility and affordability of commercially available drones has created substantial challenges for security agencies, military organisations, airport authorities, and industrial facility operators. Manual surveillance systems are demonstrably insufficient for continuous, uninterrupted monitoring, owing to inherent human limitations and the prohibitive operational cost of sustained human oversight.

Key challenges associated with drone detection in operational environments include:

- Detection of small, fast-moving aerial objects against complex and dynamic backgrounds.
- Real-time identification without significant processing latency.
- Sustained monitoring capability without operator fatigue or loss of vigilance.
- Scalability to cover large surveillance areas simultaneously.

There is, therefore, a demonstrable requirement for an automated, accurate, and real-time drone detection system capable of identifying UAVs from visual inputs with minimal human intervention and high operational reliability.

4. Project Objectives

The following objectives were defined to guide the development and evaluation of the Vyomanetra system:

1. Develop a fully functional AI-powered drone detection system.
2. Train a custom object detection model using the YOLOv8 framework on a labelled drone dataset.
3. Implement and validate drone detection on static images.
4. Implement and validate drone detection on pre-recorded video footage.
5. Implement and validate real-time drone detection using a live webcam feed.
6. Utilise GPU acceleration via CUDA to achieve efficient model training and inference.
7. Demonstrate the practical applicability of computer vision techniques in surveillance systems.
8. Establish a modular and extensible codebase suitable for future enhancement.

5. Technology Stack

The following technologies and tools were employed during the design, development, and evaluation phases of the project:

Technology / Tool	Purpose
Python 3.x	Core programming language
YOLOv8 (Ultralytics)	Object detection framework
PyTorch	Deep learning backend
OpenCV	Image and video processing
NumPy	Numerical computing
Git & GitHub	Version control and source code management
NVIDIA RTX 3050 (CUDA)	GPU-accelerated model training and inference

The YOLOv8 framework, developed by Ultralytics, was selected for its state-of-the-art detection accuracy, real-time inference capability, and comprehensive support for custom dataset training. PyTorch provided the underlying deep learning infrastructure, whilst OpenCV facilitated efficient image and video processing pipelines.

6. Dataset Description

A drone detection dataset in YOLO annotation format was utilised for model development and evaluation. All images contained bounding-box annotations specifying drone locations, with each annotation defined by class identifier, normalised centre coordinates, and bounding-box dimensions. The dataset configuration was managed through the data.yaml configuration file.

The dataset was partitioned into three standard subsets in accordance with best practices for supervised machine learning:

6.1 Training Set

The training set comprised the primary data used to optimise the model's weights and biases through iterative gradient descent. This subset exposed the model to a diverse range of drone appearances, orientations, altitudes, backgrounds, and lighting conditions.

6.2 Validation Set

The validation set was used to monitor model performance at the conclusion of each training epoch, enabling detection of overfitting and informed adjustment of training hyperparameters without introducing bias from the held-out test data.

6.3 Test Set

The test set, entirely withheld from the training process, was used to provide an unbiased final assessment of model generalisation performance on previously unseen data.

7. Methodology

The project followed a structured and systematic machine learning workflow encompassing five sequential development stages:

Stage	Phase	Description
1	**Dataset Preparation**	Download and organise dataset in YOLO directory structure; verify all image–annotation pairs prior to training.
2	**Environment Configuration**	Create a Python virtual environment; install GPU-enabled PyTorch with CUDA support for the NVIDIA RTX 3050.
3	**Model Training**	Train YOLOv8 on the prepared dataset across multiple epochs; monitor convergence via loss and mAP curves.
4	**Model Evaluation**	Assess precision, recall, and mAP on the validation set; adjust hyperparameters to prevent overfitting.
5	**Inference & Deployment**	Run inference on static images, pre-recorded video, and live webcam feeds using the trained best.pt weights.

This workflow ensured reproducibility, methodological rigour, and systematic validation of each development phase prior to progression to subsequent stages.

8. System Architecture

The Vyomanetra system architecture follows a linear processing pipeline from data ingestion through to detection output, as illustrated in the diagram below:

Input Data **Image, Video File, Live Webcam Feed**
Pre-processing **Frame extraction, resizing, normalisation**
YOLOv8 Training **Custom drone dataset, GPU-accelerated**
Trained Weights **best.pt-optimised model parameters**
Inference Engine **Real-time forward pass via YOLOv8**
Detection Output **Bounding boxes, Class labels, Confidence scores**

The pipeline is modular in design, enabling independent replacement or enhancement of individual components. The trained model weights file (best.pt) serves as the central artefact, encapsulating all learned detection parameters and enabling portability across inference environments.

9. Training Process

The YOLOv8 model was trained using the Ultralytics training pipeline with GPU acceleration provided by the NVIDIA RTX 3050 graphics processor and CUDA toolkit. Training was conducted across multiple epochs, with the following key parameters monitored throughout:

- Box loss – measures the accuracy of predicted bounding box coordinates.
- Classification loss – measures the correctness of class predictions.
- Distribution focal loss (DFL) – measures the quality of bounding box distribution regression.
- Precision and recall – evaluated on the validation set at each epoch.
- Mean Average Precision (mAP@0.5 and mAP@0.5:0.95) – primary detection accuracy metrics.

Training output graphs demonstrated progressive reduction in all loss values and consistent improvement in precision, recall, and mAP metrics throughout the training process, confirming effective model convergence. The model weights corresponding to the best validation mAP were saved as best.pt and subsequently used for all inference tasks.

10. Results and Observations

The trained Vyomanetra model was evaluated across three distinct inference modalities. Performance was assessed based on detection accuracy, bounding box precision, and system responsiveness:

Detection Mode	Performance	Key Observations
Image Detection	High	Bounding boxes accurately localised around drone targets across varied backgrounds and scales.
Video Detection	High	Frame-by-frame inference tracked drones reliably throughout pre-recorded test sequences.
Live Webcam Detection	Moderate–High	Real-time inference confirmed practical feasibility for live surveillance deployment with suitable GPU support.

10.1 Image Detection

The trained model accurately identified drone targets in static test images, generating tightly fitted bounding boxes with associated class labels and confidence scores. Detection performance was consistent across varied backgrounds, lighting conditions, and drone sizes within the test set.

10.2 Video Detection

Frame-by-frame inference on pre-recorded video footage demonstrated reliable drone tracking throughout test sequences. The system maintained consistent detection across successive frames, with bounding box positions updating accurately to reflect drone movement.

10.3 Real-Time Webcam Detection

Live inference via webcam feed confirmed the system's capability for real-time deployment. Inference latency was within acceptable bounds for surveillance applications, demonstrating the practical viability of the model for live monitoring scenarios when supported by appropriate GPU hardware.

11. Project Screenshots

The following screenshots provide visual evidence of the successful implementation, training, and deployment of the Vyomanetra Drone Detection System.

11.1 Training Results

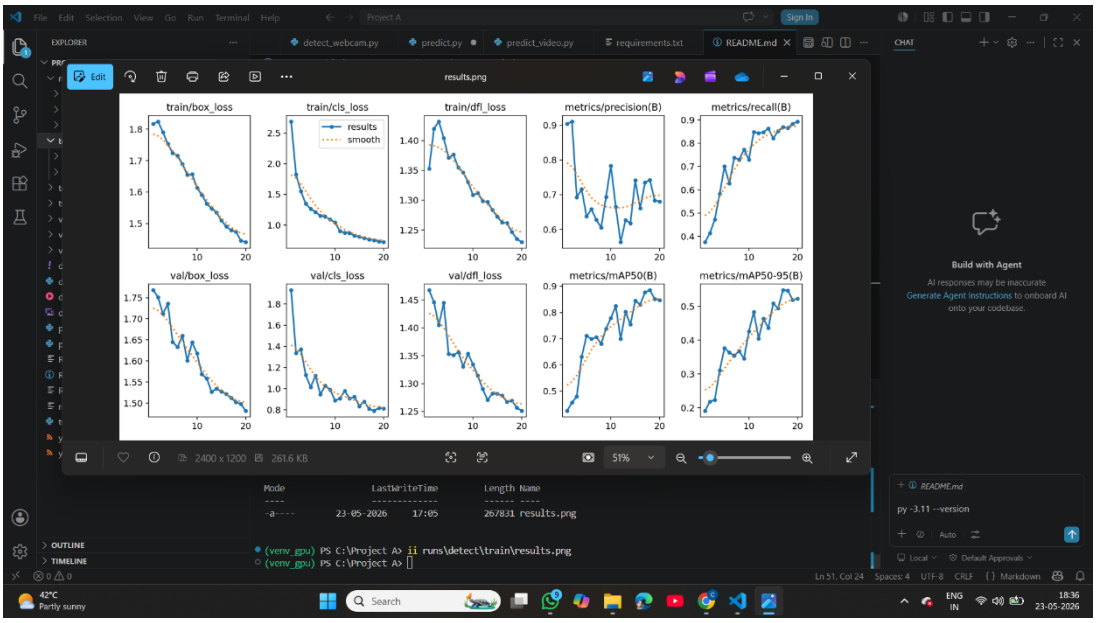

Figure 1

11.2 Image Detection Output

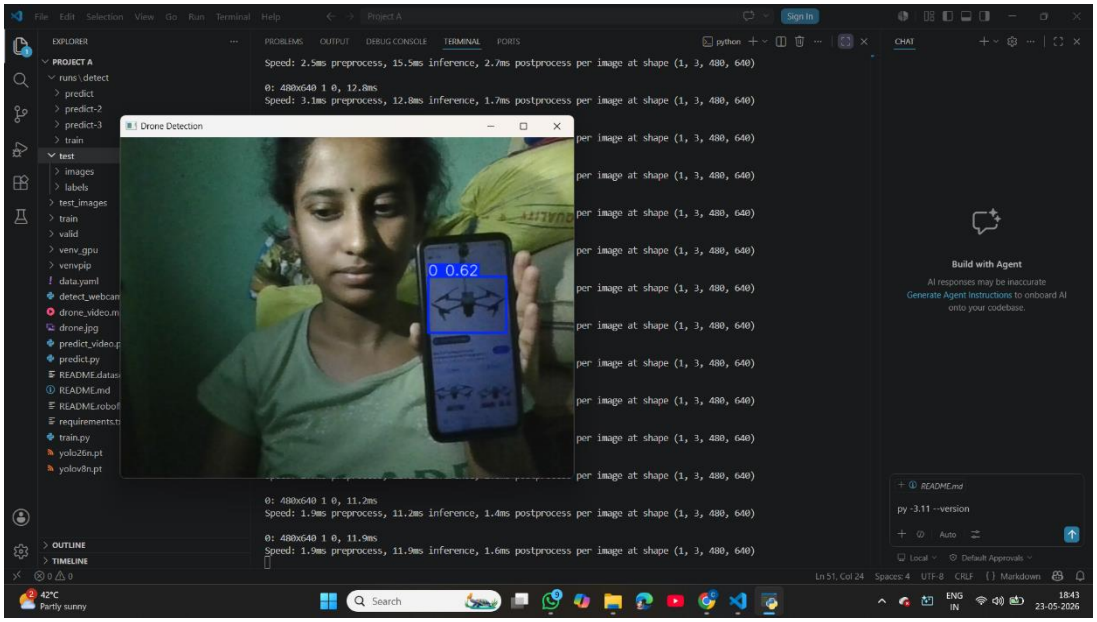

Figure 2

11.3 Video Detection Output

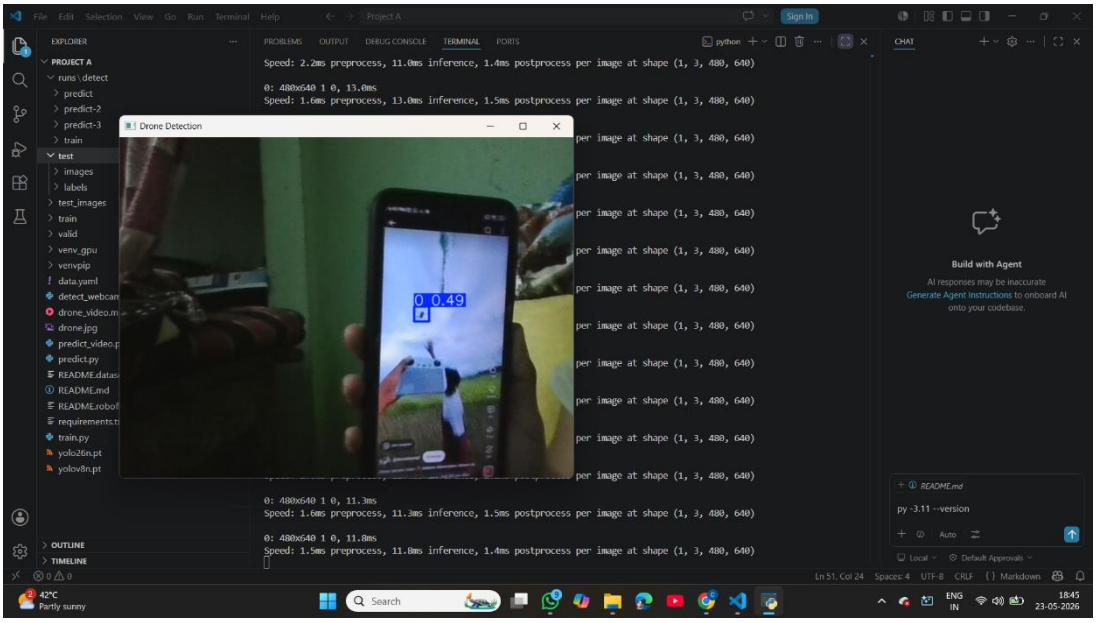

Figure 3

12. Challenges Encountered

Several technical challenges were encountered and systematically resolved during the course of project development:

- GPU Configuration: Establishing CUDA compatibility between the installed NVIDIA RTX 3050 driver, CUDA Toolkit version, and the GPU-enabled PyTorch build required careful version alignment and iterative configuration testing.
- PyTorch Installation: Selecting and installing the correct GPU-enabled PyTorch wheel for the target CUDA version necessitated consultation of the official PyTorch installation matrix and manual package specification.
- Dataset Organisation: Ensuring strict adherence to the YOLO directory structure and verifying the integrity of all image–annotation pairs required systematic validation prior to training commencement.
- Model Training Optimisation: Initial training runs required hyperparameter tuning to achieve balanced precision and recall without overfitting to the training distribution.
- Version Control Configuration: Establishing the remote GitHub repository, configuring authentication, and managing large model weight files required resolution of version control configuration issues.

Each challenge was addressed through systematic debugging, environment reconfiguration, and reference to official framework documentation, resulting in a stable and fully reproducible development environment.

13. Future Enhancements

The following enhancements are proposed to extend the capability, scalability, and operational utility of the Vyomanetra system:

- Multi-Drone Detection and Tracking: Implementation of multi-object tracking algorithms (e.g. Deep SORT, Byte Track) to simultaneously track multiple drone targets across video frames.

- Drone Type Classification: Extension of the detection model to classify drone categories (e.g. quadcopter, fixed-wing, hexacopter) in addition to detection, supporting more granular threat assessment.
- CCTV System Integration: Development of interfaces for integration with existing closed-circuit television infrastructure to enable deployment within established surveillance networks.
- Edge Deployment: Optimisation of the model for deployment on embedded and edge computing platforms (e.g. NVIDIA Jetson, Raspberry Pi with accelerator) to support low-power field installations.
- Real-Time Alert Generation: Implementation of automated alert mechanisms, including visual notifications, audio alarms, and API-based messaging, triggered upon detection events.
- Sensor Fusion: Integration with radar, acoustic, and RF signal detection sensors to create a multimodal detection pipeline with enhanced robustness against occlusion and adverse environmental conditions.
- Large-Scale Surveillance Deployment: Architecture design for distributed multi-camera deployment across wide-area surveillance environments, with centralised detection management and logging.

14. Conclusion

Vyomanetra successfully demonstrates the practical application of Artificial Intelligence and Computer Vision for automated drone detection. By employing the YOLOv8 object detection framework, Python, and GPU-accelerated deep learning, the project delivers a functional and scalable solution for UAV identification across multiple input modalities.

The system achieves reliable detection performance on static images, pre-recorded video streams, and live webcam feeds, validating the effectiveness of modern object detection architectures for real-world surveillance applications. The structured development methodology, encompassing dataset preparation, model training, evaluation, and multi-modal inference, ensures both reproducibility and extensibility.

The project establishes a robust technical foundation for future research and development in UAV detection, and demonstrates clear pathways towards industrial deployment in security, defence, and critical infrastructure protection domains.

15. References

No.	Reference	URL
1	**Ultralytics YOLOv8 Documentation**	https://docs.ultralytics.com
2	**PyTorch Documentation**	https://pytorch.org/docs
3	**OpenCV Documentation**	https://docs.opencv.org
4	**Git Documentation**	https://git-scm.com/doc
5	**GitHub Documentation**	https://docs.github.com

Vangapalli Kaveri

Drone Detection AI Project – Vyomanetra 2026

GitHub: https://github.com/Vangapalli-Kaveri/Vyomanetra